Задание 1: реляционная алгебра для r и s

```
r(X,Y) :- s(X,X), t(Y), not(s(Y,Y)).
s(X,Y) :- u(V,W), v(W,X), v(W,Y), not(X=Y).
```

EDB: t (унарное), u(V,W), v(W,X).

Выражение для s

Правило читается: объединить u и два экземпляра v через общий атрибут W, исключить пары где X=Y. Обозначим второй экземпляр v с переименованием X→Y: ρ*{X→Y}(v). Тогда естественное соединение u(V,W) ⋈ v(W,X) ⋈ ρ*{X→Y}(v)(W,Y) работает на W:

```
s = π_{X,Y}( σ_{X≠Y}( u ⋈ v ⋈ ρ_{X→Y}(v) ) )
```

Выражение для r

Правило: X такие что s(X,X), Y такие что Y ∈ t, но s(Y,Y) нет.

Определим «диагональ» s — элементы, встречающиеся на главной диагонали:

```
Diag(s) = π_{A}( σ_{A=B}(s) )       % {X | (X,X) ∈ s}
```

Тогда:

```
r = ρ_{A→X}(Diag(s))  ×  ρ_{A→Y}(t − Diag(s))
```

X берётся из Diag(s), Y — из элементов t, которых нет в Diag(s).

Задание 2: наивная и полунаивная рекурсия для even

```
bi(X,Y)   :- g(X,Y).
bi(Y,X)   :- g(X,Y).
even(X,Y) :- bi(X,Z), bi(Z,Y).
even(X,Y) :- bi(X,U), bi(U,V), even(V,Y).
```

g = {(a,b),(b,c),(c,d),(d,e),(e,f),(f,g)} — цепочка a→b→c→d→e→f→g.

bi = g ∪ π_{B,A}(g) — граф стал двунаправленным: a↔b↔c↔d↔e↔f↔g.

(a) Наивная рекурсия

$even^0 = \varnothing$

Итерация 1 (R1 даёт пары длины 2, R2 ничего — $even^0$ пуст):

bi(X,Z)	bi(Z,Y)	even(X,Y)
(a,b)	(b,a)	(a,a)
(a,b)	(b,c)	(a,c)
(b,c)	(c,b)	(b,b)
(b,c)	(c,d)	(b,d)
(c,d)	(d,c)	(c,c)
(c,d)	(d,e)	(c,e)
(d,e)	(e,d)	(d,d)
(d,e)	(e,f)	(d,f)
(e,f)	(f,e)	(e,e)
(e,f)	(f,g)	(e,g)
(f,g)	(g,f)	(f,f)
(c,b)	(b,a)	(c,a)
(d,c)	(c,b)	(d,b)
(e,d)	(d,c)	(e,c)
(f,e)	(e,d)	(f,d)

bi(X,Z)	bi(Z,Y)	even(X,Y)
(g,f)	(f,e)	(g,e)
(g,f)	(f,g)	(g,g)

```
even¹ = {(a,a),(a,c),(b,b),(b,d),(c,a),(c,c),(c,e),(d,b),
(d,d),(d,f),
          (e,c),(e,e),(e,g),(f,d),(f,f),(g,e),(g,g)}
```

17 пар — расстояния 2 (и кратные 2 через возврат).

Итерация 2 (R2: bi ⋈ bi ⋈ $even^1$):

Функция двух шагов: step2(X) = {V | ∃Z: bi(X,Z) ∧ bi(Z,V)}: step2(a)={a,c}, step2(b)={b,d}, step2(c)={a,c,e}, step2(d)={b,d,f}, step2(e)={c,e,g}, step2(f)={d,f}, step2(g)={e,g}.

Новые пары:

- X=a, V=c: $even^1$(c,·)={a,c,e} → (a,e)
- X=b, V=d: $even^1$(d,·)={b,d,f} → (b,f)
- X=c, V=e: $even^1$(e,·)={c,e,g} → (c,g)
- X=e, V=c: $even^1$(c,·)={a,c,e} → (e,a)
- X=f, V=d: $even^1$(d,·)={b,d,f} → (f,b)
- X=g, V=e: $even^1$(e,·)={c,e,g} → (g,c)

```
even² = even¹ ∪ {(a,e),(b,f),(c,g),(e,a),(f,b),(g,c)}   % 23
пары
```

Итерация 3 (R2: bi ⋈ bi ⋈ $even^2$):

- X=a, V=c: $even^2$(c,·) теперь содержит g → (a,g)
- X=g, V=e: $even^2$(e,·) теперь содержит a → (g,a)

```
even³ = even² ∪ {(a,g),(g,a)}   % 25 пар — неподвижная точка
```

Итерация 4: $\Delta even^3$ = {(a,g),(g,a)}, step2 даёт только уже известные → $\Delta even^4$ = ∅.

Финальная модель: even = все пары (X,Y) с чётным расстоянием в цепочке:

```
{a,c,e,g} × {a,c,e,g}  ∪  {b,d,f} × {b,d,f}   = 16 + 9 = 25
пар
```

(b) Полунаивная рекурсия — RA-выражения

bi вычисляется один раз:

```
bi = g ∪ π_{B→A, A→B}(g)
```

Операция «два шага через bi» применённая к некоторому отношению R:

```
Step2(R) = π_{X,Y}( ρ_{X,U}(bi) ⋈_{U} ρ_{U,V}(bi) ⋈_{V} R )
```

Инициализация:

```
even⁰  = ∅
Δeven¹ = Step2(bi) − even⁰  =  Step2(bi)        % R1: пары
длины 2
even¹  = Δeven¹
```

Шаг n+1 (n ≥ 1):

```
Δeven^{n+1} = Step2(Δeven^n) − even^n
even^{n+1}  = even^n ∪ Δeven^{n+1}
```

R1 производит одно и то же множество Step2(bi) на каждом шаге — оно уже вошло в $even^1$, новых данных не даёт. R2 расширяет even через Δeven прошлого шага. Полунаивный метод берёт только приращение Δeven^n вместо всего even^n, избегая повторного вычисления уже известных фактов.

Шаг	Наивно	Δ (полунаивно)

Шаг	Наивно	Δ (полунаивно)
1	17	17
2	23	6
3	25	2
4	25 (конец)	0 (конец)

На каждом шаге полунаивный метод джойнит bi с маленьким Δ, а не с растущим even.

Задание 3: reach с magic sets

```
reach(X,Y) :- g(X,Y).
reach(X,Y) :- g(X,Z), reach(Z,Y).
```

g = {(a,b),(b,c),(c,d),(d,e),(f,g),(g,h),(h,i),(b,i)}

Граф: два потока — a→b→{c→d→e, i} и f→g→h→i.

(a) Неэффективность полунаивного метода для ?reach(c,Y)

Полунаивный метод вычисляет все пары (X,Y) ∈ reach, даже если нас интересует только X=c.

Δreach^n	Новые пары
Δ^1 = g	(a,b),(b,c),(c,d),(d,e),(f,g),(g,h),(h,i),(b,i) — 8 пар
Δ^2	(a,c),(a,i),(b,d),(c,e),(f,h),(g,i) — 6 пар
Δ^3	(a,d),(b,e),(f,i) — 3 пары
Δ^4	(a,e) — 1 пара
Δ^5	∅ — конец

Итого: 18 пар для всех узлов. Из них нас интересуют только (c,d) и (c,e) — 2 пары. Метод посетил все узлы графа, включая f,g,h,i и a,b, не имеющих отношения к запросу.

(b) Top-down вычисление ?reach(c,Y)

Цель: `reach(c,Y)`.

```
reach(c,Y)
 ├─ R1: g(c,Y) → Y=d          → reach(c,d) ✓
 └─ R2: g(c,Z), reach(Z,Y)
         └─ Z=d: reach(d,Y)
                  ├─ R1: g(d,Y) → Y=e    → reach(d,e) ✓
                  └─ R2: g(d,Z), reach(Z,Y)
                         └─ Z=e: reach(e,Y)
                                  ├─ R1: g(e,Y) → нет рёбер
→ ∅
                                  └─ R2: нет g(e,?) → ∅
```

?reach(c,Y) = {d, e}

Посещены только узлы c, d, e — ничего лишнего.

(c) Magic sets для ?reach(c,Y)

```
m(c).
m(Z) :- m(X), g(X,Z).

reach(X,Y) :- m(X), g(X,Y).
reach(X,Y) :- m(X), g(X,Z), reach(Z,Y).
```

Вычисление m (какие узлы «нужны» для запроса):

```
m⁰ = {c}
m¹: m(c), g(c,d) → m(d).   m = {c,d}
m²: m(d), g(d,e) → m(e).   m = {c,d,e}
m³: m(e), g(e,?) → нет.    Неподвижная точка: m = {c,d,e}
```

Вычисление reach с ограничением m(X):

- m(c), g(c,d) → reach(c,d)
- m(d), g(d,e) → reach(d,e)

- m(e), g(e,?) → ничего
- m(c), g(c,d), reach(d,e) → reach(c,e)

```
reach = {(c,d), (d,e), (c,e)}
?reach(c,Y) = {d, e}
```

Узлы a,b,f,g,h,i не тронуты. Вместо 18 пар — 3 пары.

(d) * Magic sets для ?reach(c,e)

Запрос: может ли c достичь e? (оба аргумента связаны)

Magic set такой же — распространяем от c вперёд:

```
m(c).
m(Z) :- m(X), g(X,Z).

reach(X,Y) :- m(X), g(X,Y).
reach(X,Y) :- m(X), g(X,Z), reach(Z,Y).
```

m = {c,d,e} (как в (c)). Вычисляем reach ⊆ {c,d,e}×{...}, потом проверяем (c,e) ∈ reach.

Получаем reach(c,e) = **TRUE**.

Можно также распространять от e назад, чтобы остановиться раньше, но для данного размера графа это излишне.

(e) ** Magic sets для ?reach(X,e)

Запрос: все X, из которых достижима e (Y связан, X свободен).

Распространяем назад от e по обратным рёбрам:

```
m(e).
m(X) :- m(Z), g(X,Z).    % если Z нужен и есть ребро X→Z, то
X тоже нужен

reach(X,Y) :- m(Y), g(X,Y).
reach(X,Y) :- m(Y), g(X,Z), reach(Z,Y).
```

Вычисление m (обратное распространение):

```
m⁰ = {e}
m¹: g(X,e) → g(d,e) → m(d).     m = {e,d}
m²: g(X,d) → g(c,d) → m(c).     m = {e,d,c}
m³: g(X,c) → g(b,c) → m(b).     m = {e,d,c,b}
m⁴: g(X,b) → g(a,b) → m(a).     m = {e,d,c,b,a}
m⁵: g(X,a) → нет.    Неподвижная точка: m = {a,b,c,d,e}
```

Узлы f,g,h,i в m не попали — e из них недостижима.

Вычисление reach с m(Y)=e:

- reach(X,e) :- m(X), g(X,e) → g(d,e) → reach(d,e)
- reach(c,e) :- m(c), g(c,d), reach(d,e) ✓
- reach(b,e) :- m(b), g(b,c), reach(c,e) ✓
- reach(a,e) :- m(a), g(a,b), reach(b,e) ✓

```
?reach(X,e) = {a, b, c, d}
```

f, g, h, i не рассматривались — экономия на нерелевантной части графа.